

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА ІМЕНІ
ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 2

з навчальної дисципліни

"ТЕХНОЛОГІЇ РОЗРОБКИ КОРПОРАТИВНИХ WEB-ДОДАТКІВ"

Виконала:

ст.гр. 402-ТН

Чернушенко Валерія Юріївна

Перевірив:

Тютюнник Петро Богданович

Полтава 2026

Завдання для виконання:

Завдання

1. Обґрунтування вибору технологічного стеку (Technology Stack) На основі вимог, зібраних у першій лабораторній роботі, оберіть технології для реалізації проєкту.

- **Створіть документ або розділ у звіті, де чітко визначте:**
 - **Мова програмування Back-end:** (наприклад, C#/.NET, Java/Spring, Node.js, Python/Django).
 - **Фреймворк Front-end:** (наприклад, React, Angular, Vue.js або Server-Side Rendering — Razor/Thymeleaf).
 - **Система керування базами даних (СКБД):** (PostgreSQL, MS SQL, MongoDB, Redis тощо).
 - **Інструменти DevOps/Hosting:** (Docker, GitHub Actions, Azure/AWS — опціонально).
- **Надайте обґрунтування:** Чому саме ці технології підходять для вирішення вашої бізнес-задачі? (Наприклад: "Обрано MongoDB, оскільки структура даних каталогу товарів є гнучкою та часто змінюється" або "Обрано .NET через надійну типізацію та швидкість розробки корпоративних API").

2. Архітектурне моделювання за методологією C4 (C4 Model) Спроектуйте архітектуру вашої системи, створивши три рівні діаграм C4. Це допоможе зрозуміти структуру системи від загального до конкретного.

- **Рівень 1: System Context Diagram (Контекст).**
 - Покажіть вашу систему як одну "чорну скриньку" в центрі.
 - Відобразіть усіх користувачів (Actors) та зовнішні системи (наприклад, платіжні шлюзи, API поштових розсилок), з якими вона взаємодіє.
- **Рівень 2: Container Diagram (Контейнери).**
 - "Відкрийте" систему і покажіть її основні технічні блоки (контейнери).

- Це можуть бути: Single Page Application, Mobile App, API Application, Database, File System.
- Вкажіть технології для кожного контейнера та протоколи взаємодії між ними (HTTP/HTTPS, TCP, GRPC).
- **Рівень 3: Component Diagram (Компоненти).**
 - Деталізуйте **один** основний контейнер (наприклад, API Application).
 - Покажіть внутрішні компоненти (наприклад: AuthController, EmailService, UserRepository) та їхні зв'язки.

3. Проєктування бази даних (ER-Model / Data Schema) Спроєктуйте схему зберігання даних. Вибір типу діаграми залежить від обраної СКБД:

- **Варіант А: Реляційна база даних (SQL)**
 - Побудуйте класичну **ER-діаграму (Entity-Relationship Diagram)**.
 - Вкажіть сутності, атрибути, первинні (PK) та зовнішні (FK) ключі.
 - Визначте типи зв'язків (1:1, 1:N, M:N).
 - Забезпечте нормалізацію даних (мінімум до 3-ї нормальної форми).
- **Варіант Б: Нереляційна база даних (NoSQL)**
 - Створіть діаграму або схему колекцій/документів.
 - Відобразіть структуру JSON-документів для основних сутностей.
 - Вкажіть стратегію зв'язків: **Embedding** (вкладення даних) чи **Referencing** (посилання на ID).
 - Обґрунтуйте вибір стратегії (наприклад, "Використано Embedding для коментарів, щоб отримати пост разом із коментарями за один запит").

Очікуваний результат (Звіт):

1. Таблиця або список обраного тех стеку з обґрунтуванням (1-2 абзаци).
2. Три діаграми C4 (Context, Container, Component) з коротким описом до кожної.
3. Схема бази даних (ER-діаграма або схема документів NoSQL).

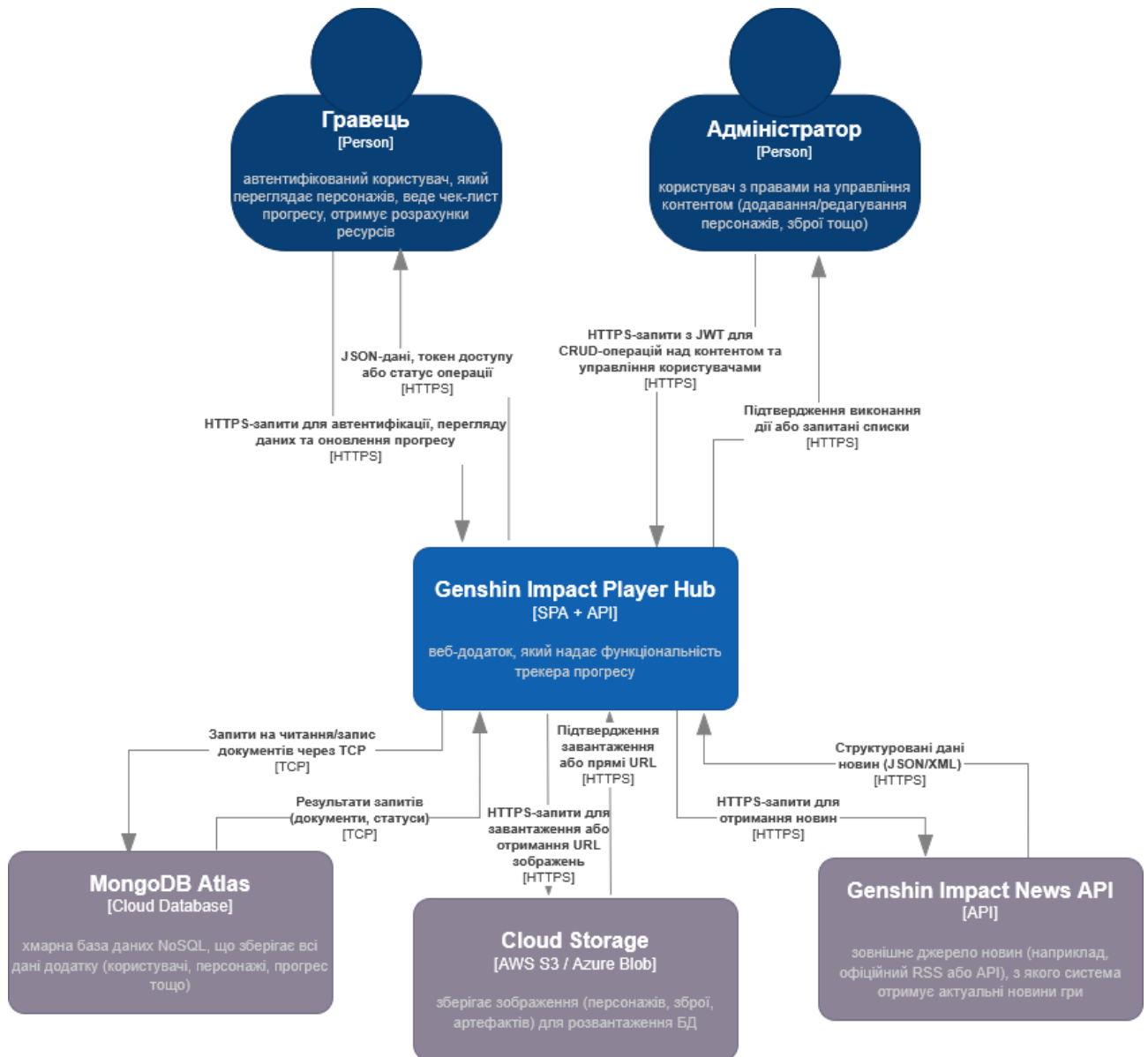
Обґрунтування вибору технологічного стеку

Компонент	Обрана технологія	Обґрунтування
Back-end	C# / .NET 8 (ASP.NET Core)	.NET Core забезпечує високу продуктивність, надійну типізацію, що важливо для коректної роботи зі складною бізнес-логікою (розрахунок ресурсів, управління прогресом). Вбудована підтримка Dependency Injection, Entity Framework Core та JWT-автентифікації спрощує розробку корпоративних API. Також .NET має велику екосистему та стабільність, що відповідає вимогам надійності (SLA 99.5%).
Front-end	React	React дозволяє створити інтерактивний односторінковий додаток (SPA) з багатою взаємодією (чек-листи, фільтри, розрахунки). TypeScript підвищує надійність коду. React має велику кількість готових компонентів (наприклад, для сіток, форм), що пришвидшує розробку. Адаптивність (НФВ-08) легко реалізується за допомогою CSS-фреймворків.
База даних	MongoDB	Структура даних про персонажів та матеріали часто змінюється (додаються нові типи матеріалів, атрибути). MongoDB дозволяє зберігати документи з різними полями без міграцій. Прогрес гравця (чек-лист) має гнучку вкладену структуру, що зручно зберігати як один документ на користувача. Висока масштабованість та простота розробки при роботі з JSON-подібними даними.

Обраний стек забезпечує швидку розробку, масштабованість (горизонтальне масштабування API через stateless-дизайн), а також відповідає вимогам безпеки та продуктивності.

Архітектурне моделювання за методологією C4

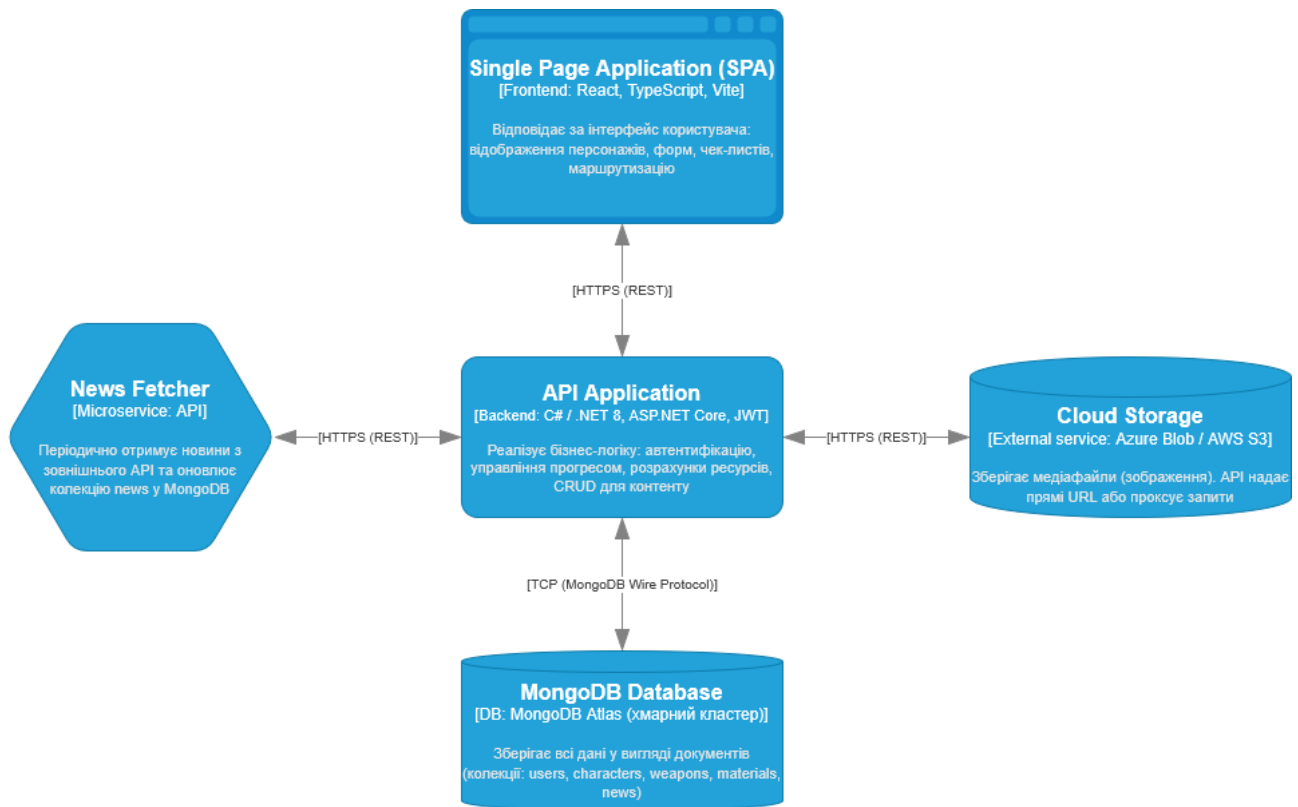
System Context Diagram



Діаграма показує систему «Genshin Impact Player Hub» як єдиний блок («чорну скриньку») та її взаємодію із зовнішніми користувачами та системами. Центральний елемент – веб-додаток. Основними акторами є **Гравець** (перегляд інформації, ведення чек-листу) та **Адміністратор** (управління контентом). Зовнішні системи: **MongoDB Atlas** (хмарна база даних), **Cloud Storage**

(зберігання зображень) та **Genshin Impact News API** (джерело новин). Взаємодії відбуваються через HTTPS (з користувачами та зовнішніми API) та TCP (з MongoDB Atlas).

Container Diagram

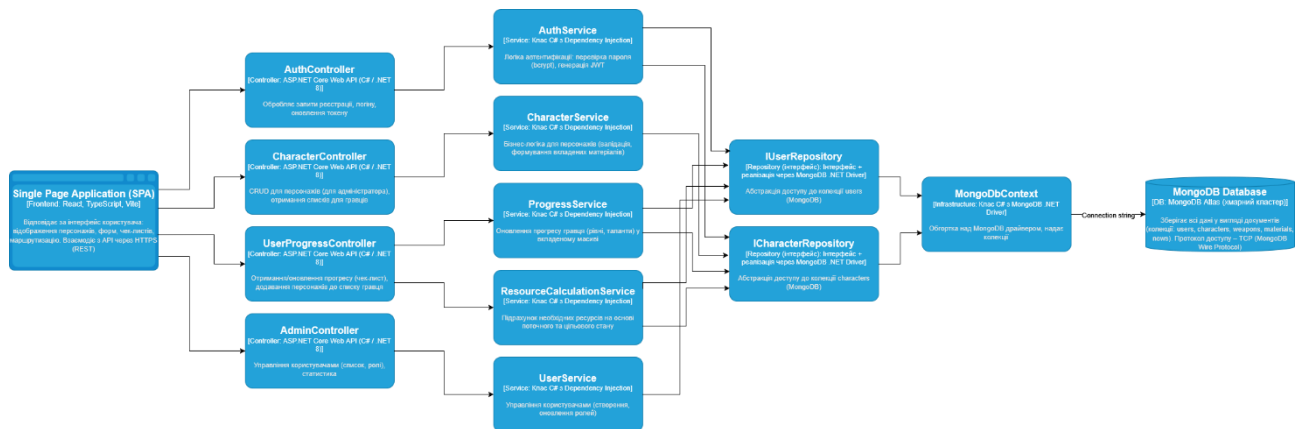


Діаграма розкриває внутрішню структуру системи, показуючи основні технічні блоки (контейнери) та протоколи зв'язку між ними.

- **Single Page Application (SPA)** – клієнтська частина на React, взаємодіє з API через HTTPS.
- **API Application** – серверна частина на ASP.NET Core (C#), реалізує бізнес-логіку, автентифікацію (JWT) та доступ до даних.
- **MongoDB Database** – хмарний кластер MongoDB Atlas, зберігає всі дані у вигляді документів.
- **Cloud Storage** – хмарне сховище (Azure Blob / AWS S3) для зображень.
- **News Fetcher** (опціонально) – мікросервіс для періодичного отримання новин із зовнішнього API.

Протоколи: HTTPS (SPA ⇔ API, API ⇔ Cloud Storage, API ⇔ News API),
TCP (API ⇔ MongoDB Atlas).

Component Diagram



Діаграма деталізує внутрішню структуру контейнера **API Application**.

Основні компоненти:

- **Контролери** (AuthController, CharacterController, UserProgressController, AdminController) – обробляють HTTP-запити, викликають сервіси.
- **Сервіси** (AuthService, CharacterService, ProgressService, ResourceCalculationService, UserService) – містять бізнес-логіку.
- **Репозиторії** (IUserRepository, ICharacterRepository) – абстракції доступу до MongoDB.
- **MongoDbContext** – обгортка над MongoDB .NET Driver, надає колекції.

Технології: ASP.NET Core Web API, MongoDB .NET Driver, JWT, BCrypt, AutoMapper, вбудований DI та логування. Компоненти організовані за принципом «контролер => сервіс => репозиторій», що забезпечує розділення відповідальності.

Проектування бази даних (NoSQL – MongoDB)

Оскільки обрано MongoDB, ми створимо **три основні колекції** зі стратегією вкладення (embedding) для зв'язаних даних, що часто читаються разом, та посилань (referencing) для незалежних сутностей.

Колекція characters (персонажі)

Зберігає всю інформацію про персонажа, включаючи вкладені матеріали (щоб отримувати все одним запитом).

```
{
  "_id": ObjectId("..."),
  "name": "Diluc",
  "element": "Pyro",
  "rarity": 5,
  "weaponType": "Claymore",
  "imageUrl": "https://...",
  "materials": [
    {
      "materialId": ObjectId("..."), // посилання на матеріал в окремій колекції
      "name": "Agnidus Agate Sliver",
      "category": "boss",
      "quantityPerLevel": 1
    },
    {
      "materialId": ObjectId("..."),
      "name": "Small Lamp Grass",
      "category": "local_specialty",
      "quantityPerLevel": 3
    }
    // інші матеріали
  ]
}
```

Чому embedding? – Матеріали персонажа використовуються майже завжди разом з інформацією про персонажа. Вкладення дозволяє отримати повний профіль за один запит. Оскільки матеріали можуть змінюватися, ми зберігаємо їх як масив вкладених документів. Для зменшення дублювання назв, ми додаємо

поле `materialId` (посилання), але назву та категорію дублюємо для швидкого доступу.

Колекція `weapons` (зброя)

Окрема колекція, оскільки зброя не завжди потрібна разом з персонажем.

```
{
  "_id": ObjectId("..."),
  "name": "Wolf's Gravestone",
  "type": "Claymore",
  "rarity": 5,
  "imageUrl": "https://...",
  "materials": [ ... ] // аналогічно вкладені матеріали
}
```

Колекція `users` (користувачі з прогресом)

Зберігає дані автентифікації та вкладений масив прогресу по персонажах. Це дозволяє одним запитом отримати весь прогрес гравця.

```
{
  "_id": ObjectId("..."),
  "email": "player@example.com",
  "passwordHash": "$2a$11$...",
  "role": "player",
  "createdAt": ISODate("2026-03-29T10:00:00Z"),
  "progress": [
    {
      "characterId": ObjectId("..."), // посилання на персонажа
      "characterName": "Diluc",      // дублюється для швидкості
      "currentLevel": 80,
      "targetLevel": 90,
      "talents": {
        "auto": 8,
```

```

    "skill": 8,
    "burst": 8
  },
  "owned": true
},
{
  "characterId": ObjectId("..."),
  "characterName": "Kaeya",
  "currentLevel": 60,
  "targetLevel": 80,
  "talents": {
    "auto": 4,
    "skill": 5,
    "burst": 4
  },
  "owned": true
}
]
}

```

Чому embedding? – Прогрес гравця завжди використовується разом з даними користувача. Вкладення дозволяє уникнути окремої колекції user_characters та складних JOIN-запитів. Оновлення прогресу відбувається атомарно на рівні одного документа користувача.

Колекція materials (довідник матеріалів)

Окрема колекція для незалежного управління матеріалами.

```

{
  "_id": ObjectId("..."),
  "name": "Agnidus Agate Sliver",
  "category": "boss",

```

```
"imageUrl": "https://..."  
}
```

Колекція news (новини)

Окрема колекція для кешу новин.

```
{  
  "_id": ObjectId("..."),  
  "title": "Version 4.5 Update",  
  "content": "...",  
  "publishedAt": ISODate("2026-03-25T12:00:00Z"),  
  "sourceUrl": "https://..."  
}
```

Обґрунтування вибору стратегій

Сутність	Стратегія	Причина
Матеріали персонажа	Embedding (з частковим дублюванням)	Матеріали завжди потрібні разом з персонажем, тому краще зберігати їх у тому ж документі. Дублювання назви та категорії дозволяє уникнути додаткового запиту до колекції materials при відображенні.
Прогрес гравця	Embedding	Увесь прогрес одного гравця зберігається в одному документі, що забезпечує швидкий доступ та атомарні оновлення.
Зброя	Окрема колекція	Зброя використовується незалежно від персонажів, але може містити вкладені матеріали (аналогічно).
Матеріали (довідник)	Окрема колекція	Централізоване зберігання матеріалів з можливістю редагування без зміни документів персонажів.

Така схема забезпечує високу продуктивність читання (менше запитів) і відповідає шаблонам проєктування NoSQL.

Відповіді на контрольні запитання

1. Що таке модель C4 і для чого потрібен кожен з її рівнів (Context, Container, Component)?

Модель C4 — це спосіб візуалізації архітектури програмного забезпечення на чотирьох рівнях абстракції.

- a) **Context (контекст)** показує систему як чорну скриньку, її користувачів і зовнішні системи. Дає загальне уявлення про те, хто і що взаємодіє з системою.
- b) **Container (контейнери)** розкриває систему, показуючи основні технологічні блоки (веб-додаток, API, база даних) та протоколи зв'язку між ними. Допомогає зрозуміти, як розподілені обов'язки на високому рівні.
- c) **Component (компоненти)** деталізує один із контейнерів, показуючи внутрішні компоненти (контролери, сервіси, репозиторії) та їхні зв'язки. Дає змогу оцінити модульність і розподіл відповідальності.

2. У чому принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних? Коли варто обирати кожен з них?

Реляційні (SQL): використовують таблиці з фіксованою схемою, підтримують ACID-транзакції, складні зв'язки, мову SQL. Обирають, коли потрібна строга узгодженість, складні запити з JOIN, дані мають чітку структуру (наприклад, банківські системи, CRM).

NoSQL (MongoDB): використовує документи JSON, не потребує фіксованої схеми, масштабується горизонтально. Обирають, коли дані мають гнучку структуру, потрібна висока швидкість запису, або коли зручно агрегувати дані в один документ (наприклад, профіль гравця з його прогресом).

3. Що таке теорема CAP (Consistency, Availability, Partition tolerance) і як вона впливає на вибір бази даних?

CAP-теорема стверджує, що в розподіленій системі можна забезпечити лише два з трьох властивостей:

- **Consistency (узгодженість)** – всі вузли бачать одні дані одночасно.

- **Availability (доступність)** – кожен запит отримує відповідь (успішну або ні).
- **Partition tolerance (стійкість до розподілу)** – система продовжує працювати при втраті зв'язку між вузлами.

Вибір БД залежить від пріоритетів: MongoDB за замовчуванням налаштована на CP (Consistency + Partition tolerance) у межах одного репліка-сету, але може бути сконфігурована на AP для розподілених кластерів. Це впливає на вибір: для нашого додатку важливіша узгодженість даних прогресу, тому залишаємо налаштування за замовчуванням.

4. Які переваги та недоліки мікросервісної архітектури порівняно з монолітною (на рівні Container diagram)?

Моноліт: простіший у розробці та деплої, але складніший у масштабуванні та може стати “вузьким місцем” при зростанні навантаження. У нашому проєкті, враховуючи обмежений обсяг, монолітний API з чітким розділенням на компоненти (як у діаграмі) є оптимальним вибором.

Переваги мікросервісів:

- Незалежне масштабування окремих сервісів.
- Можливість використання різних технологій для різних сервісів.
- Локальність змін (зменшує ризик впливу на всю систему).
- Покращена відмовостійкість (падіння одного сервісу не зупиняє весь додаток).

Недоліки:

- Складність розробки (міжсервісна комунікація, розподілені транзакції).
- Додаткові витрати на DevOps (оркестрація, моніторинг).
- Затримки через мережеві виклики.